System Design: Latency

Easy English lecture notes | Topic: response time, monolithic vs distributed architecture, caching, CDN and performance improvements.

1. What is latency?

Latency is the time a user waits for a request to receive a response. Example from the lecture: a user enters facebook.com, the request travels to a server, the server processes it, and the response travels back.

Lecture model: Total latency = T1 (request network travel) + T2 (response network travel) + T3 (server computation). In short: **network delay + processing delay**.

```
User  -- request (T1) -->  Server
User  <-- response (T2) --  Server
                        processing (T3)
```

2. Monolithic vs distributed architecture

Monolithic application: Modules are typically deployed together. Calls between modules in the same process can avoid additional network round trips.

Distributed application: Modules/services may run separately. Calls between services add network communication and can increase end-to-end latency.

Architecture	Typical internal communication	Latency consideration
Monolith	In-process method calls	Usually fewer internal network hops
Distributed	Network calls between services	Extra network hops may add delay

Important nuance: Distributed systems do not always have higher overall user latency. Parallel processing, placement near users, and scaling can offset network overhead. The lecture highlights the added cost of inter-service calls.

3. Method 1: Caching

Caching stores a reusable result so repeated requests can be answered without repeating expensive computation or querying the original data source. The lecture gives the example of a function that returns the same result for the same user ID.

Flow: Check cache for a key (for example, user ID). On a cache hit, return the saved result. On a cache miss, compute or fetch the result, store it with an appropriate expiry, then return it.

```
Request -> Cache lookup
           |-- HIT  -> Return saved result
           `-- MISS -> Compute / database -> Save -> Return
```

Example: A profile endpoint repeatedly requests details for user 101. A cache can store the profile for a short period to reduce repeated work. Cached data needs expiration or invalidation when it changes.

4. Method 2: CDN (Content Delivery Network)

A CDN uses geographically distributed edge servers to deliver content closer to users. In the lecture, a user in India accesses static content hosted far away; a nearby CDN edge can reduce delivery time.

Common CDN content: images, JavaScript, CSS, video segments and other cacheable assets. CDN edge locations can serve cached content without fetching it from the distant origin on every request.

5. Caching vs CDN

Caching	CDN
General technique of saving reusable data/results	Distributed delivery network with edge locations
May exist in browser, application, Redis or other layers	Often caches content near geographically dispersed users
Reduces repeated computation or data fetching	Reduces distance and origin traffic for cacheable content

Clarification: The lecture contrasts server-side caching with a geographically nearby CDN. In practice, caching is not limited to the application server, and CDNs themselves use caching.

6. Method 3: Improve server hardware

More capable CPUs, sufficient RAM, faster storage and adequate network resources may reduce processing time when hardware is the bottleneck. Measure performance first; hardware upgrades do not fix every cause of latency.

7. Java / Spring Boot example

In a Spring Boot service, caching can avoid repeatedly loading a user profile. The example below is illustrative; a cache provider and caching configuration are required.

```
@Service
public class UserService {
    @Cacheable(value = "profiles", key = "#userId")
    public UserProfile getProfile(Long userId) {
        return repository.findById(userId)
                .orElseThrow();
    }
}
```

Enable caching with **@EnableCaching** and configure a suitable provider. When a profile changes, update or evict stale cache entries.

8. Interview questions

Q: What is latency?
Elapsed time between making a request and receiving its response.

Q: Why can distributed systems have extra latency?
Service-to-service network calls add transmission and coordination overhead.

Q: How does caching reduce latency?
It returns stored results rather than repeating expensive work.

Q: What does a CDN do?
It serves cacheable content from edge locations closer to users.

Q: How is a CDN different from caching?
Caching is a general technique; a CDN is a geographically distributed delivery infrastructure that often uses caching.

9. Quick revision

Latency = network delay + processing delay. Monoliths commonly use in-process calls; distributed systems can add service-to-service network calls. Three lecture techniques: **caching, CDN, better hardware**. Also measure database time, network hops and cache hit rate before optimizing.

These notes reorganize the supplied Hindi transcript into simple English. Technical clarifications are explicitly labeled where the lecture simplified a concept.